

CXR Redis Lab Manual
SW.14 — Redis + Redis Insight + cache-aside
Full implementation record

CXR Ops Lab — `cxr-ops-lab`

2026-05-31

Abstract

Complete bootcamp record for **SW.14**: every file added for **Redis (:6379)**, **Redis Insight (:5540)**, the **cache-aside** shell demo, how keys connect to **Claim Studio (:3000)** conceptually, and troubleshooting from the live lab session (empty browser, 127.0.0.1 vs redis, TTL expiry, second-run DEL).

Contents

1	What Redis does in CXR (vs other observe labs)	2
2	Architecture: cache-aside end-to-end	3
2.1	Data flow	3
2.2	Cache-aside steps (syllabus)	3
2.3	Cached key and payload	3
3	Docker stack (compose.redis.yaml)	4
3.1	Services	4
3.2	Why two ports?	4
4	Redis Insight — connect and browse (highlight)	5
4.1	Start stack	5
4.2	Open UI in Cursor	5
4.3	Add database (first time only)	5
4.4	Golden path in UI	5
4.5	Longer TTL for screenshots	6
5	Step-by-step procedure (CLI + UI)	7
5.1	Step 1 — Start Redis + Insight	7
5.2	Step 2 — Cache-aside demo	7
5.3	Step 3 — Smoke test	7
5.4	Step 4 — Manual redis-cli (optional)	7
5.5	Step 5 — Evidence	7
5.6	Step 6 — Stop (optional)	7
6	Files created and touched (full inventory)	8
6.1	New files (SW.14)	8
6.2	Related (not modified for Redis wiring)	8
7	Environment variables	9
8	Troubleshooting (from live session)	10
9	ADR: where Redis fits CXR production	11
10	Port map (CXR lab)	12
11	Study plan linkage	13
12	Live adjustments (Redis Insight baked in)	14
13	UI evidence — Redis Insight (:5540)	15

Chapter 1

What Redis does in CXR (vs other observe labs)

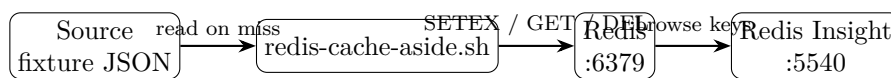
Pattern	Tool	URL	SW
Metrics	Grafana	:3001	SW.10
Traces	Jaeger	:16686	SW.11
Logs	Kibana	:5601	SW.12
Async events	Kafka / Kafka UI	:9092 / :8082	SW.13
Hot read cache	Redis + Insight	:6379 / :5540	SW.14

Claim Studio (<http://localhost:3000/claim-studio>) is the product dashboard. **Redis is not embedded** in the Next.js app in this lab — a standalone script demonstrates **cache-aside** against a static JSON artifact that matches the Kafka event schema.

Chapter 2

Architecture: cache-aside end-to-end

2.1 Data flow



2.2 Cache-aside steps (syllabus)

1. GET `cxr:claim:analyzed:{claim_id}` — miss returns `(nil)`.
2. Load `lab/fixtures/claim-analyzed.demo.json` (stand-in for DB/API after analyze).
3. SETEX key TTL json — default TTL **300s**.
4. GET — cache hit on repeat read.
5. **Invalidation:** DEL key on update (second script run when key is warm).

Mermaid source: `docs/diagrams/06-redis-cache-aside.mmd`.

2.3 Cached key and payload

Item	Value
Key	<code>cxr:claim:analyzed:demo-1</code>
Type	STRING
Fixture	<code>lab/fixtures/claim-analyzed.demo.json</code>
Schema	<code>schemas/cxr.claim-analyzed.v1.json</code>
TTL env	<code>CXR_CACHE_TTL_SEC</code> (default 300)

Chapter 3

Docker stack (`compose.redis.yaml`)

3.1 Services

Service	Image	Role
<code>redis</code>	<code>redis:7.4-alpine</code>	Cache server; published :6379 ; no persistence (<code>-save ""</code> , no AOF)
<code>redis-insight</code>	<code>redis/redisinsight:2.7.0</code>	Web UI :5540 ; volume <code>redis_insight_data</code>

3.2 Why two ports?

Port	Protocol	Browser?
6379	Redis (RESP)	No — use <code>redis-cli</code> or Insight
5540	HTTP (Insight UI)	Yes — Simple Browser / Chrome

Chapter 4

Redis Insight — connect and browse (highlight)

4.1 Start stack

```
cd cxx-ops-lab
./scripts/17-redis-up.sh
./lab/redis-cache-aside.sh
```

4.2 Open UI in Cursor

1. **Ctrl+Shift+P** → **Simple Browser: Show**
2. URL: <http://localhost:5540>
3. Allow ~30s on first boot if smoke warns “Insight not ready”.

4.3 Add database (first time only)

Wrong (common): Connection URL `redis://default@127.0.0.1:6379`
Insight runs *inside* Docker; 127.0.0.1 is Insight’s own container, not Redis.

Correct:

```
redis://redis:6379
```

Or **Connection Settings:** Host `redis`, Port `6379`, alias `cxx-lab`.

Linux fallback if needed: `redis://host.docker.internal:6379` (Redis published on host :6379).

4.4 Golden path in UI

1. Run `./lab/redis-cache-aside.sh` **once** (creates key).
2. Insight → database `redis:6379` → **db0** → refresh.
3. Click `cxx:claim:analyzed:demo-1` — JSON + TTL on right pane.
4. Screenshot: `evidence/SW14-redis-insight-2026-05-31.png`

Do not use “Load sample data” — unrelated demo keys.

4.5 Longer TTL for screenshots

```
CXR_CACHE_TTL_SEC=3600 ./lab/redis-cache-aside.sh
```


Chapter 5

Step-by-step procedure (CLI + UI)

5.1 Step 1 — Start Redis + Insight

```
./scripts/17-redis-up.sh
```

5.2 Step 2 — Cache-aside demo

```
./lab/redis-cache-aside.sh          # miss -> load -> SETEX -> hit
./lab/redis-cache-aside.sh          # warm key -> DEL (invalidation)
```

5.3 Step 3 — Smoke test

```
./scripts/17-redis-smoke.sh
```

Expect: PONG, Insight :5540 (or WARN wait 30s), CACHE HIT.

5.4 Step 4 — Manual redis-cli (optional)

```
docker exec -it cxr-ops-lab-redis-1 redis-cli ping
docker exec -it cxr-ops-lab-redis-1 redis-cli GET cxr:claim:analyzed:
demo-1
docker exec -it cxr-ops-lab-redis-1 redis-cli TTL cxr:claim:analyzed:
demo-1
```

5.5 Step 5 — Evidence

- evidence/SW14-redis-verify-2026-05-31.md — all checkboxes
- evidence/SW14-redis-insight-2026-05-31.png — Insight with JSON visible

5.6 Step 6 — Stop (optional)

```
docker compose -f compose.redis.yaml down
```

Chapter 6

Files created and touched (full inventory)

6.1 New files (SW.14)

File	Purpose
compose.redis.yaml	Redis + Redis Insight compose
lab/redis-cache-aside.sh	Cache-aside demo (GET/SETEX/DEL)
lab/fixtures/claim-analyzedCXR.sh	Redis Insight cache payload
scripts/17-redis-up.sh	compose.redis.yaml up -d; prints :6379 and :5540
scripts/17-redis-smoke.sh	PING, Insight probe, cache-aside check
scripts/build-redis-manual-PDF.sh	Builds this PDF
docs/CXR-REDIS-LAB-MANUAL.md	Markdown twin
docs/CXR-REDIS-LAB-MANUAL.tex	LaTeX source
docs/CXR-REDIS-LAB-MANUAL.pdf	Printable record
docs/diagrams/06-redis-cache-aside.md	Sequence diagram
evidence/SW14-redis-verify-2026-05-31.txt	Y026-05-31 list
evidence/SW14-redis-insight-2026-05-31.png	U2026-05-31.png

6.2 Related (not modified for Redis wiring)

File	Note
schemas/cxr.claim-analyzed.Samirson	Same shape as SW.13 Kafka sample
compose.yaml / cxr-ui	Claim Studio :3000 — not calling Redis yet
docs/PLATFORM-SYLLABUS-MAP.SW	SW.14 row updated

Chapter 7

Environment variables

Variable	Default	Effect
CXR_CACHE_CLAIM_ID	demo-1	Key suffix
CXR_CACHE_TTL_SEC	300	SETEX seconds
CXR_CACHE_ARTIFACT	fixture path	JSON source file

Chapter 8

Troubleshooting (from live session)

- **Browser on :6379 fails** — expected; Redis is not HTTP. Use :5540 or `redis-cli`.
- **Insight connect fails with 127.0.0.1** — use host `redis` or URL `redis://redis:6379`.
- **Key in list but “does not exist”** — TTL expired or second script run DEL'd key; re-run `cache-aside` once; refresh UI.
- **Empty db0 / 0 keys** — same; run `./lab/redis-cache-aside.sh`; refresh; avoid running script twice in a row before screenshot.
- **Smoke WARN Insight not ready** — wait 30s; reload <http://localhost:5540>.
- **Container filter** — scripts target `cxr-ops-lab-redis-1` (not `redis-insight`).

Chapter 9

ADR: where Redis fits CXR production

Good cache candidates: reference codes, static policy snippets, *completed* analyze snapshots with TTL.

Risky: caching in-flight **Run Analysis** without invalidation — stale approvals/denials.

This lab: script-only; no Next.js ioredis on :8251 or :3000.

Chapter 10

Port map (CXR lab)

Port	Service	SW.14
3000	Claim Studio	Source of truth (not cached yet)
6379	Redis	Cache API
5540	Redis Insight	Web UI
8251	Rehearsal dev	Unchanged
8082	Kafka UI	SW.13
5601	Kibana	SW.12
16686	Jaeger	SW.11

Chapter 11

Study plan linkage

Item	Maps to
SW.14 syllabus row	Cache-aside + TTL + invalidation
User verify	<code>SW14-redis-verify-2026-05-31.md</code>
After SW.14	SW.15+ electives

Chapter 12

Live adjustments (Redis Insight baked in)

- Added `redis-insight` service to `compose.redis.yaml` — UI on **:5540** (Redis :6379 has no browser UI).
- Insight connect URL `redis://redis:6379` — **not** `127.0.0.1` (Insight runs inside Docker network).
- Scripts filter container `cxr-ops-lab-redis-1` — avoids matching `redis-insight` container.
- Second `redis-cache-aside.sh` run **DELs** warm key — re-run once before UI screenshot.
- `CXR_CACHE_TTL_SEC=3600` optional for long screenshot windows.

Chapter 13

UI evidence — Redis Insight (:5540)

Key `cxr:claim:analyzed:demo-1` with CXR JSON payload and TTL.

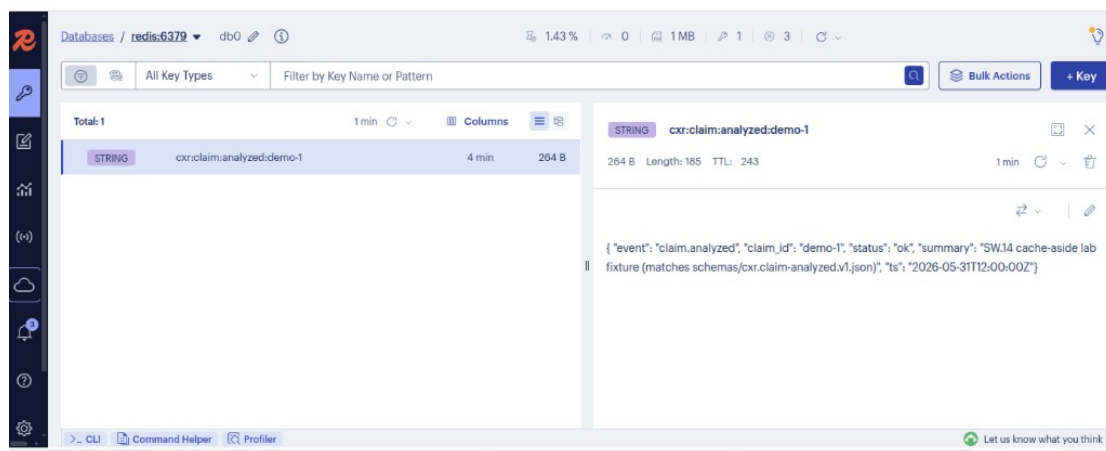


Figure 13.1: Redis Insight — cached claim-analyzed JSON (SW.14 evidence).